BIGQUERY THEORY Q&A

BY - SHUBHAM WADEKAR

Copyright © Shubham Wadekar. All rights reserved.

This material is for personal use only. No part may be copied, shared, resold, or published without written permission. Unauthorized distribution is strictly prohibited and may result in action.

For permissions or licensing, contact: [shubham.p.wadekar@gmail.com]

Disclaimer: This content is for educational purposes. Accuracy is attempted but not guaranteed.

No job outcome is promised.

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

1. How does BigQuery's serverless model simplify infrastructure management compared to traditional databases?

In a traditional database, I have to manage servers, set up clusters, worry about scaling, and handle backups or patches. That takes a lot of time and effort from engineers.

BigQuery is serverless, so Google manages all of that for me. I don't need to provision machines or think about how many nodes I need for a query. It automatically scales up when I run heavy workloads and scales down when idle, so resources are used efficiently. Backups, replication, and patching are also handled in the background.

For me as a data engineer, this means I can focus on the SQL and business logic, not infrastructure. I don't have to spend time tuning servers or worrying about downtime. That's a big productivity gain compared to traditional databases.

2. Why is separating storage and compute beneficial in BigQuery?

Separating storage and compute gives flexibility and cost efficiency. In BigQuery, data is stored in Colossus (Google's distributed storage), and compute happens through slots in the Dremel execution engine.

This separation helps in two ways:

- I can store massive amounts of data cheaply without tying it to compute. Even if the data is petabytes, storage is just sitting there at low cost.
- Compute can scale independently. If I run a big query, BigQuery allocates thousands of slots to finish it quickly. If I'm not running queries, I don't pay for compute at all (in on-demand pricing).

This model avoids the problem in traditional systems where storage and compute scale together. In BigQuery, they scale independently, which makes it more efficient and cost-effective.

3. Why does BigQuery use a columnar storage format, and how does it help analytics?

BigQuery uses columnar storage because most analytics queries read only a few columns across a huge number of rows. If data were stored row by row, the engine would waste time reading all the unnecessary columns.

With columnar storage, BigQuery only reads the specific columns needed for the query. This reduces the amount of data scanned, which makes queries faster and cheaper. Columnar format also compresses better because similar values are stored together, which further improves performance.

For example, if I want to calculate the average salary from an employee table, BigQuery only scans the salary column, not all the other fields like name, address, or department. That's why columnar storage is such a good fit for analytical workloads.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

4. What role does the Dremel engine play in BigQuery query execution?

The Dremel engine is the core query execution engine behind BigQuery. When I run a SQL query, Dremel breaks it down into a tree of execution stages and distributes those tasks across thousands of workers in parallel. Each worker scans a portion of the data from storage, processes it, and sends results up the tree until the final result is produced.

What makes Dremel powerful is that it works in a massively parallel and interactive way, so even very large queries scanning terabytes or petabytes can return results in seconds. It also works well with BigQuery's columnar storage, which means it only reads the columns required by the query.

In simple terms, Dremel makes BigQuery's serverless SQL experience possible I just write SQL, and under the hood Dremel orchestrates distributed query execution at scale.

5. How does BigQuery automatically scale resources when query workloads increase?

BigQuery is serverless, so it handles scaling without me provisioning anything. When workloads increase say I submit a very large query or many users run queries at once BigQuery automatically allocates more slots (compute resources) to handle the load. These slots can come from thousands of CPUs spread across Google's infrastructure.

If queries are smaller or fewer, BigQuery uses fewer slots, so resources are not wasted. I don't have to think about adding nodes, resizing clusters, or tuning scaling is automatic.

In on-demand pricing, scaling is transparent and I only pay for the data processed. In flat-rate reservations, scaling depends on the slots I've purchased, and BigQuery distributes those across workloads. Either way, I don't have to manually manage scaling like I would in traditional databases.

6. What are two major differences between BigQuery and a traditional RDBMS or Hadoop-based system?

The first major difference is the architecture. Traditional RDBMS systems are designed for OLTP workloads row-based storage, lots of small updates, and transaction consistency. Hadoop-based systems require me to manage clusters and distribute data manually. BigQuery, on the other hand, is serverless and designed for OLAP it uses columnar storage, automatic sharding, and the Dremel engine for distributed queries. I don't manage any infrastructure.

The second difference is the cost and scaling model. In traditional systems or Hadoop, scaling requires adding hardware or managing clusters, which takes time and money. In BigQuery, compute and storage are separated, and scaling is automatic. I pay per query (on-demand) or by reserving slots, which is much simpler and often more cost-efficient.

So, compared to RDBMS or Hadoop, BigQuery is serverless, columnar, and elastic which makes it much better suited for large-scale analytics without operational overhead.

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

7. What types of partitioning are supported in BigQuery?

BigQuery supports a few types of partitioning that let me split a large table into smaller, manageable pieces:

1. Ingestion-time partitioning: BigQuery automatically partitions the table based on the time the data was ingested. This is good when I don't have a date column but still want time-based partitions.
2. Column-based partitioning: I choose a DATE, TIMESTAMP, or DATETIME column, and BigQuery partitions the table using values from that column. This is the most common type for example, partitioning a sales table by sale_date.
3. Integer range partitioning: Instead of dates, I can partition on an integer column by defining a start, end, and interval. For example, partitioning customers by customer_id ranges (1–1000, 1001–2000, etc.).

These options give me flexibility. Time-based partitioning is great for logs and events, while integer range partitioning works for other types of large datasets where values fall into ranges.

8. Why is it important to use partition filters in BigQuery queries?

Partition filters are critical because they control how much data BigQuery scans. Without a partition filter, BigQuery might scan the entire table, which can be terabytes, making the query slower and more expensive.

For example, if I have a partitioned table by sale_date and I only need last month's data, I should filter like:

WHERE sale_date BETWEEN '2025-08-01' AND '2025-08-31'

This way, only those partitions are scanned.

Partition filters also improve query performance because BigQuery prunes unnecessary partitions early in the query execution. In fact, some partitioned tables can even enforce a requirement for partition filters to avoid accidental full scans.

So, using partition filters is about both cost savings and performance optimization.

9. How does clustering in BigQuery improve query performance?

Clustering organizes the data within each partition (or the whole table if not partitioned) based on the values of one or more columns. Rows with similar values are stored close together.

This improves performance because when I run a query with filters on those clustered columns, BigQuery only scans the blocks of data that match the filter instead of scanning the whole table or partition.

For example, if I cluster a sales table by customer_id, then a query filtering on a specific customer will only scan a small portion of the data, even inside a big partition.

Another benefit is that clustering improves aggregation performance, since grouped values are stored together. Clustering is also automatically maintained by BigQuery as data is added, so I don't need to manually reorganize the table.

In short, clustering makes queries faster and cheaper when I often filter or aggregate on the same columns.

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

10. Can you combine partitioning and clustering in BigQuery? If yes, how?

Yes, we can absolutely combine partitioning and clustering, and in fact it's a best practice for very large tables. Partitioning reduces the data by time or integer ranges, and clustering organizes the data inside each partition.

For example, I can partition a sales table by sale_date (daily partitions) and then cluster within each partition by customer_id and region. That way, queries that filter by date only scan relevant partitions, and queries that also filter by customer or region only scan the relevant blocks inside those partitions.

In SQL, I define it like this:

```
CREATE TABLE my_dataset.sales_partitioned_clustered

PARTITION BY DATE(sale_date)

CLUSTER BY customer_id, region AS

SELECT * FROM my_dataset.sales_raw;
```

So, partitioning handles the "big cut" of the data, and clustering handles the "fine cut" inside those partitions.

11. When would you prefer clustering over partitioning?

I'd prefer clustering over partitioning in cases where:

- The column I filter on has too many unique values to be a good partition key (for example, customer_id). Partitioning works best when the number of partitions is relatively low, not millions.
- I already have a partition (usually on a date), but I want faster queries when filtering or aggregating on another column, like region, customer_id, or product_category.
- I want to reduce query cost and improve performance for selective filters, but partitioning alone won't give me enough savings.

In short: if the filter column has high cardinality or is used often in queries but doesn't make sense as a partition key, clustering is the better choice.

12. How does clustering help with data pruning inside partitions?

Clustering physically organizes rows with similar values together inside a partition. BigQuery also creates metadata about the value ranges stored in each block.

When I run a query with a filter on the clustered column, BigQuery checks the metadata and only scans the blocks that match the filter. This is called block-level pruning.

For example, if a partition contains 1 million rows clustered by customer_id, and I query for customer_id = 12345, BigQuery won't scan the full partition it will jump directly to the blocks containing that customer.

This saves a lot of time and reduces the amount of data scanned, which directly lowers query costs.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

13. Why should SELECT * be avoided in BigQuery queries?

In BigQuery, every query cost depends on how many bytes are scanned. When I use SELECT *, BigQuery scans every single column in the table, even if I only need one or two of them.

For example, imagine I have a customer table with 50 columns and a terabyte of data. If my analysis only needs customer_id and region, but I use SELECT *, BigQuery will still read all 50 columns, which could scan 10x or 20x more data than necessary. That directly increases query cost and slows execution because BigQuery has to process more data than needed.

Another issue is schema changes if new columns are added in the future, SELECT * will pull them too, which may break queries or produce unexpected results.

That's why in production or repeated queries, the best practice is to always select only the columns I need. This makes the query cheaper, faster, and more stable. I might use SELECT * only for quick exploration or debugging when cost is not a concern.

14. How do materialized views improve performance in BigQuery?

Materialized views are special views that physically store the results of a query, unlike normal views which just store the query definition. BigQuery automatically refreshes materialized views when the base table changes, so they're always up to date.

The performance gain comes from the fact that queries on a materialized view don't need to scan the entire base table. They read the precomputed results instead.

For example, say I have a transactions table with 10 billion rows, and I frequently run queries like "total sales per day." Without a materialized view, every query would scan all 10 billion rows and re-aggregate them. With a materialized view, BigQuery stores those daily totals, so when I query, it just reads a few thousand rows of aggregated data.

This makes queries much faster and cheaper, especially for dashboards or recurring reports where the same aggregations are needed repeatedly. Materialized views are most effective for queries with predictable patterns, like filtering and grouping, but less useful for highly dynamic ad-hoc queries.

15. When would you use APPROX_COUNT_DISTINCT instead of COUNT(DISTINCT)?

COUNT(DISTINCT col) gives the exact number of unique values, but it can be very expensive on huge datasets because BigQuery has to shuffle and compare all distinct values across partitions. That can slow queries down and increase cost.

APPROX_COUNT_DISTINCT(col) uses an algorithm called HyperLogLog++ to estimate the number of unique values. The result is approximate but very close to the real count, usually within a small error margin. The big advantage is that it's much faster and uses fewer resources.

For example, if I want to know the number of unique visitors to a website over the last year, and the table has billions of events, running COUNT(DISTINCT user_id) might be very slow and costly. Using APPROX_COUNT_DISTINCT(user_id) would give me an answer in seconds at a fraction of the cost, and the tiny error margin (maybe 1%) is acceptable for business analysis.

So I'd use APPROX_COUNT_DISTINCT when:

- The dataset is very large.
- The exact number isn't critical (like reporting, monitoring, or trend analysis).
- I want to save on performance and cost.

© Shubham Wadekar

But if I need 100% accuracy (like billing, compliance, or financial reporting), I'd still use COUNT(DISTINCT).

16. How does query caching reduce cost in BigQuery?

BigQuery automatically caches the results of queries for 24 hours. If I run the same query again and the underlying tables haven't changed, BigQuery will return the cached result instantly instead of re-scanning the data.

This reduces cost because cached queries don't scan any data BigQuery simply returns the saved result for free. It also improves performance since results come back much faster.

For example, if a dashboard refreshes multiple times a day with the same query, only the first run scans data and incurs cost. Later runs within 24 hours just read the cached result at no charge.

In practice, caching is very helpful for repeated queries, BI dashboards, and testing. But if I want to force BigQuery to ignore the cache (for example, if I suspect data changed), I can disable the cache in the query settings.

So query caching is both a cost-saving feature and a way to speed up repeated queries.

17. What does a dry run do in BigQuery?

A dry run in BigQuery is a way to validate and preview a query without actually running it. It doesn't scan any data or return results instead, it tells me how much data the query would scan and whether the syntax is valid.

This is very useful because query cost depends on bytes scanned. With a dry run, I can estimate cost before execution and adjust the query if it's scanning too much.

For example:

```
bq query --dry_run --use_legacy_sql=false 'SELECT * FROM dataset.big_table WHERE date > "2025-01-01"'
```

BigQuery will return something like: "This query will process 1.2 TB." I might then realize I should filter by partition to scan only 50 GB.

In practice, I often use dry runs when working with large tables to make sure my query is efficient and won't accidentally cost too much.

18. What is the main difference between on-demand pricing and flat-rate slot reservations in BigQuery?

In on-demand pricing, I pay per query based on the amount of data scanned. This is flexible because if I run fewer queries, I pay less. It's good for unpredictable or small workloads. But if I run a lot of heavy queries, the cost can spike.

In flat-rate pricing (slot reservations), I purchase a fixed number of slots (compute resources) that my queries will use. I pay a fixed monthly fee regardless of how much I query. This gives predictable costs and is better for large, steady workloads or when multiple teams share BigQuery.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

So the main difference is:

- On-demand = variable cost, pay per data scanned.
- Flat-rate = fixed cost, pay for reserved compute slots.

In practice, I've seen companies start with on-demand, and once query volume grows, switch to flat-rate for cost predictability.

19. Why is denormalization a common schema design pattern in BigQuery?

BigQuery is optimized for analytical workloads, not transactional ones. It's often more efficient to denormalize data meaning store data in fewer, wider tables instead of normalizing it into many small tables like in OLTP databases.

The reason is that BigQuery is designed to scan large tables quickly. Joins across many tables can be expensive because they require shuffling data across slots. By denormalizing, I can avoid excessive joins and make queries simpler and faster.

For example, instead of keeping separate orders, customers, and products tables and joining them every time, I might create a denormalized sales table that includes customer and product details directly. Storage is cheap in BigQuery, and repeated joins are expensive, so this trade-off makes sense.

That said, denormalization should be balanced with maintainability sometimes partial normalization still makes sense for reusable dimensions.

20. Why are shuffle operations expensive in BigQuery queries?

Shuffling happens when BigQuery has to redistribute data between workers to complete a query stage. This usually occurs during joins, aggregations, and ORDER BY operations.

For example, if I do a join on customer_id, BigQuery may need to shuffle rows with the same customer_id across different slots so they can be processed together. Similarly, large GROUP BY or DISTINCT queries can trigger shuffling.

Shuffling is expensive because it moves large amounts of data across the network between slots. This increases execution time and cost.

To minimize shuffling, I can:

- Use partitioning and clustering so that data is already organized.
- Reduce data early with filters before joins.
- Use approximate functions like APPROX_COUNT_DISTINCT instead of DISTINCT on massive datasets.
- Denormalize tables where joins are frequent.

In short, shuffle operations are costly because they involve heavy data movement across BigQuery's distributed infrastructure.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

21. What is the advantage of using nested and repeated fields in BigQuery schemas?

Nested and repeated fields let me represent hierarchical or one-to-many relationships directly inside a single BigQuery table. Instead of splitting data into multiple tables and doing joins, I can store them together in one record.

For example, a customer might have multiple orders. In a traditional schema I'd have a customers table and an orders table, then join them. In BigQuery, I can make orders a repeated nested field inside the customers table. That way, all of a customer's orders are stored with their profile in one row.

The advantage is fewer joins, which improves query performance and makes the schema more natural for semi-structured data (like JSON). It also reduces costs because BigQuery scans fewer tables. That's why nested/repeated fields are common when ingesting data from APIs or logs that already come in JSON format.

22. Why is denormalization often preferred over normalization in BigQuery?

In OLTP databases, normalization is preferred because it reduces redundancy and keeps data consistent across transactions. But in BigQuery, joins across large tables are expensive they require shuffling data across slots.

Denormalization avoids that by storing related information together in fewer, wider tables. Since BigQuery is optimized for scanning huge tables quickly, it's usually cheaper and faster to scan a wide table than to repeatedly join multiple normalized tables.

For example, instead of having separate orders, customers, and products tables, I might create a sales table with all customer and product info included. Storage in BigQuery is cheap, but compute is expensive so denormalization trades some extra storage for faster, cheaper queries.

23. What is the difference between NULLABLE and REQUIRED fields in BigQuery?

When I define a schema in BigQuery, each field has a mode:

- NULLABLE means the field can be empty. Rows can have a NULL value for that column. This is the default mode.
- REQUIRED means the field must always have a value. NULLs are not allowed.

In practice, I use NULLABLE for most fields, especially when the source data might not always provide a value. REQUIRED is only used when I'm certain every row will have that field for example, an order_id or timestamp that is always present.

Choosing REQUIRED can help enforce data quality, but I usually prefer NULLABLE for flexibility because many ingestion pipelines deal with incomplete data.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

24. What type of schema changes are allowed in BigQuery without recreating the table?

BigQuery is flexible with schema evolution. I can make certain changes without recreating the table:

- Add new columns (as long as they are NULLABLE or REPEATED).
- Relax an existing column from REQUIRED to NULLABLE.
- Change column descriptions, labels, or policy tags.

But I cannot drop columns, rename columns, or change a column's data type directly for those, I usually need to create a new table and reload data.

So schema changes in BigQuery are generally additive or relaxing, not destructive.

25. How does BigQuery handle evolving JSON data in ingestion pipelines?

When I ingest JSON into BigQuery, sometimes the schema evolves new fields appear, some fields are missing, or field types change. BigQuery handles this flexibly if I design for it:

- Missing fields are simply stored as NULLs.
- Extra fields not in the schema can be ignored during load, or I can enable schema auto-detection to pick them up.
- If new fields are added, I can update the schema to include them (as NULLABLE).
- For deeply nested JSON, BigQuery maps objects to STRUCTs and arrays to REPEATED fields, so I can still query them with SQL.

In production, I often land JSON in Cloud Storage first, then use schema evolution strategies like schema relaxation (making fields NULLABLE) or adding new columns when the JSON changes.

26. What is schema relaxation in BigQuery?

Schema relaxation means changing a field's mode from REQUIRED to NULLABLE. This makes the schema more flexible by allowing NULL values for that field.

For example, if I originally defined customer_email as REQUIRED but later discover some customers don't have emails, I can relax the field to NULLABLE. After that, ingestion won't fail when the value is missing.

Schema relaxation is helpful in real-world pipelines because source data often has missing or optional fields. Instead of rejecting the row, BigQuery just stores NULL.

So schema relaxation is all about making a strict schema more permissive without losing existing data.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

27. Why is Parquet or Avro preferred over CSV when loading data into BigQuery?

I'd say the main reason is efficiency and richer schema support.

- Parquet and Avro are columnar/binary formats that compress better and load faster into BigQuery. CSV is plain text, so it's larger, slower to parse, and costs more to store and load.
- Schema handling: Parquet and Avro store schema information within the file. This means BigQuery can automatically understand data types. CSV doesn't everything looks like text, so schema inference is less reliable.
- Support for nested/repeated fields: JSON, Parquet, and Avro handle complex schemas. CSV cannot represent nested or repeated fields naturally.
- Cost efficiency: Because Parquet and Avro are compressed and optimized, fewer bytes are read/written, which lowers costs compared to CSV.

So in practice, I always prefer Parquet or Avro for production pipelines. CSV is fine for quick manual uploads or small datasets, but not for serious ingestion.

28. What are the cost implications of streaming data into BigQuery compared to batch loading?

Streaming and batch loading have different pricing models.

- Batch loading (from GCS in Avro/Parquet/CSV) is free I only pay for storage and queries. It's cost-efficient and good for large, periodic loads.
- Streaming inserts cost extra BigQuery charges per MB inserted via the streaming API. It's small per MB, but at scale it adds up.
- Streaming also bypasses caching, so it can increase query costs since data in streaming buffers isn't always optimized immediately.

So the implication is: streaming is more expensive but gives near real-time ingestion. Batch is cheaper but adds latency. I usually recommend streaming only when fresh data is truly needed, and use batch for everything else.

29. Why is Dataflow commonly used between Pub/Sub and BigQuery?

Pub/Sub gives me real-time event data, but usually that raw stream isn't clean or ready for analytics. Dataflow sits in between to do real-time ETL before writing to BigQuery.

- It can parse and transform JSON or Avro messages.
- It can enrich the data (for example, join with reference datasets).
- It can handle deduplication, windowing, and late-arriving data.
- It can batch small messages into larger groups to optimize BigQuery streaming inserts.

Without Dataflow, raw Pub/Sub messages may not be usable directly in BigQuery. So Dataflow is the natural bridge that makes real-time ingestion reliable, clean, and cost-effective.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

30. What is the drawback of querying external data via federated queries compared to native BigQuery tables?

Federated queries let me query external sources (like GCS, Bigtable, or Google Sheets) without loading data into BigQuery. That's convenient, but there are trade-offs:

- Performance: Queries are slower because BigQuery has to reach out to the external system instead of using optimized native storage.
- Cost: More data might be scanned than necessary, and optimizations like partition pruning or clustering don't apply.
- Functionality: Some advanced BigQuery features (like materialized views, caching, or clustering) don't work on federated sources.
- Stability: Queries depend on the external system's availability and permissions.

In practice, I use federated queries for ad-hoc analysis or small datasets, but for production I load the data into native BigQuery tables.

31. How does the BigQuery Storage Write API improve streaming ingestion?

The Storage Write API is the newer, more efficient way to stream data into BigQuery compared to the older streaming insert API.

Improvements include:

- Higher throughput: It supports parallelized, batched writes from multiple clients, so it can handle much larger streaming volumes.
- Exactly-once delivery: It prevents duplicates by using stream offsets, which makes ingestion more reliable.
- Schema evolution: It can handle dynamic schema updates more gracefully.
- Better error handling: It allows partial success responses, so bad rows don't block the whole batch.

In short, the Storage Write API makes streaming ingestion faster, cheaper, and more reliable for enterprise-scale use cases.

32. What's the difference between schema-on-read and schema-on-write in BigQuery?

- Schema-on-write: The schema is enforced when data is loaded. BigQuery requires a defined schema for tables, and data must match it when inserted. If fields don't match, the load can fail unless schema evolution is allowed.
- Schema-on-read: This is used with external sources or semi-structured formats (like JSON). BigQuery interprets the schema at query time. For example, when querying external JSON files in GCS, I can parse fields on the fly using JSON_EXTRACT.

So BigQuery primarily uses schema-on-write for native tables, but it also supports schema-on-read when working with federated queries or JSON data. Schema-on-write ensures consistency; schema-on-read provides flexibility.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

33. What's one best practice for improving ingestion performance in BigQuery?

One best practice is to batch data before loading instead of sending many small files or rows individually.

- If I use streaming inserts, grouping messages into larger batches improves throughput and reduces cost. The Storage Write API helps with this.
- If I use batch loads from GCS, it's better to load fewer, larger files (e.g., 100MB–1GB each) instead of thousands of tiny files. BigQuery parallelizes large file loads much more efficiently.
- Using efficient file formats like Avro or Parquet also improves ingestion performance since they're compressed and columnar.

So the key idea is: avoid trickling in tiny records or files, and ingest in optimized batches. This reduces overhead, improves load speeds, and makes queries faster because data lands in BigQuery in an optimized way.

34. Give one example where you would use a window function in BigQuery.

A good example is when I want to calculate the running total of sales for each customer over time. Let's say I have a sales table with customer_id, order_date, and amount. If I want to see how the spending of each customer grows with each order, I can use the SUM() function as a window function with PARTITION BY customer_id and ORDER BY order_date. This way, instead of just getting the total sales per customer, I can see the cumulative amount after each purchase. Window functions are useful in these cases because they let me do calculations across rows, without grouping the whole table down to fewer rows.

35. What is PERCENTILE_CONT used for in BigQuery?

PERCENTILE_CONT is used when I want to find the exact value at a certain percentile in a dataset, like the 50th percentile (median) or 90th percentile. It's a continuous percentile function, meaning it interpolates between values if the exact rank doesn't exist. For example, if I'm analyzing website response times and want to know the 95th percentile latency, PERCENTILE_CONT will give me that value. This is important in cases like performance monitoring, where averages can be misleading, but percentiles show the actual distribution of data.

36. Why do you need UNNEST when working with arrays in BigQuery?

In BigQuery, arrays are stored as a single field containing multiple values. If I want to query or filter the individual elements inside the array, I can't directly access them like normal rows. UNNEST is needed to flatten the array into rows so that each element can be treated like a separate record. For example, if I have a column called product_ids that contains an array of multiple products in an order, using UNNEST lets me expand that array and join or filter on each product. Without UNNEST, I'd only see the array as a whole and wouldn't be able to work with individual elements effectively.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

37. What's the difference between JSON_VALUE and JSON_EXTRACT in BigQuery?

The main difference is in the type of result they return. JSON_VALUE is used when I want to extract a single scalar value, like a string, number, or boolean, from a JSON object. It's meant for simple values. On the other hand, JSON_EXTRACT returns a JSON object or JSON array as text. For example, if I have a JSON column and I only want the user's name, JSON_VALUE will give me just the string like "Shubham". But if I want the whole address field that is nested as an object, JSON_EXTRACT will give me the entire JSON structure of that address. So, JSON_VALUE is for single values, JSON_EXTRACT is for bigger JSON fragments.

38. When would you use a JavaScript UDF in BigQuery?

I would use a JavaScript UDF when I have logic that can't be easily expressed in standard SQL. For example, if I need to do complex string parsing, advanced text manipulation, or custom calculations that SQL functions don't directly support, I can write that logic in JavaScript and call it as a function inside my query. A practical case would be parsing a complicated log format, or applying some regex transformations that are easier in JavaScript. It's powerful but I'd use it carefully, because JavaScript UDFs can be slower and more expensive compared to native SQL functions.

39. What is the purpose of a semi-join in BigQuery?

A semi-join is used when I want to filter rows from one table based on whether they have a matching value in another table, but I don't actually need to return the columns from the second table. For example, if I have a customers table and an orders table, and I want to find all customers who have placed at least one order, I can use a semi-join. It checks for existence of a match but doesn't bring extra data. In BigQuery, this is usually expressed with EXISTS or IN. The purpose is efficiency I only return what I need from the first table while still applying the condition from the second table.

40. What type of metadata can you query from INFORMATION_SCHEMA in BigQuery?

INFORMATION_SCHEMA in BigQuery is like a set of system tables where I can query metadata about my datasets, tables, jobs, and other resources. Instead of manually checking in the console, I can write SQL queries against these views.

Some of the main types of metadata I can query are:

- **Dataset-level metadata**: Things like which tables exist in a dataset, when they were created, who owns them, and their size. For example, querying INFORMATION_SCHEMA.TABLES gives me table names, row counts, and storage usage.
- **Column-level metadata**: Using INFORMATION_SCHEMA.COLUMNS, I can see details like column names, data types, whether they are nullable, and their positions. This is useful if I want to audit schema across multiple tables.
- **Job metadata**: I can query INFORMATION_SCHEMA.JOBS or JOBS_BY_PROJECT to see queries that were run, who ran them, when they started and finished, how much data they processed, and whether they failed or succeeded. This is very helpful for performance monitoring or cost tracking.
- **Reservation and capacity metadata**: In enterprise setups, I can check slots usage, reservation assignments, and related information for workload management.

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

So, the types of metadata cover datasets, tables, columns, routines, jobs, reservations, and storage usage. In real life, I've used INFORMATION_SCHEMA mostly for auditing queries, checking table sizes, and tracking job costs. It's a powerful way to manage BigQuery without leaving SQL.

41. What are query stages in BigQuery execution details?

When BigQuery executes a query, it doesn't just run it as one big block. It breaks the query into smaller units called stages. Each stage represents a part of the execution plan, such as scanning data, shuffling results, joining tables, aggregating values, or writing the output.

For example, if I run a query that selects data from two large tables, applies a filter, and then does a join followed by an aggregation, BigQuery will create multiple stages:

- One stage to scan each table.
- Another stage to filter the rows.
- Then a shuffle stage to prepare data for joining.
- Another stage for the join itself.
- A final stage for the aggregation and output.

Each stage can have multiple parallel workers running tasks across different slots. That's how BigQuery achieves massive parallelism. In the execution details view in the console, I can see these stages, the amount of data processed in each stage, and whether there was a bottleneck.

Understanding stages is useful when troubleshooting slow queries for example, if I see one stage taking most of the time, I know where the bottleneck lies (like a large join or shuffle). It gives me insight into how BigQuery internally optimizes and distributes my SQL.

42. Name one common reason why a BigQuery query runs slower than expected.

One of the most common reasons is processing too much unnecessary data. For example, if my query scans entire tables with billions of rows when I only need a subset of columns or a limited date range, BigQuery has to scan terabytes instead of just gigabytes. This directly impacts query time and cost.

A classic mistake is using SELECT * on a very large table, even when I only need 3 or 4 columns. Another is forgetting to partition or cluster tables properly, so every query scans the whole dataset instead of just the relevant partitions.

For instance, if I have a logs table partitioned by date, but my query doesn't filter on the date column, BigQuery will read all partitions. That can make a query run minutes instead of seconds.

Other reasons exist too, like skewed joins (one big table being shuffled unevenly), too many complex subqueries, or excessive use of JavaScript UDFs. But the number one reason I've personally seen is scanning far more data than required because of missing filters or poor schema design.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

43. How can audit logs help in monitoring BigQuery access?

Audit logs are very important for security and governance in BigQuery. They record every action taken on BigQuery resources, like datasets, tables, and jobs. By looking at these logs, I can monitor who is accessing data, what kind of queries they are running, and whether anything suspicious is happening.

There are different types of audit logs:

- Admin activity logs: Show changes to BigQuery resources, like when someone creates, deletes, or modifies a dataset or table.
- Data access logs: Capture who read data or metadata, for example if a user queried a sensitive table.
- System event logs: Show system-driven activities.
- Policy denied logs: Show when someone tried to do something they didn't have permission for.

In real projects, these logs help with compliance. For example, if my company has sensitive customer data, I can use audit logs to check which users or service accounts queried that data and when. They also help detect unusual patterns like if suddenly a user is exporting large volumes of data at midnight, I'd catch it in the audit logs.

In practice, audit logs are stored in Cloud Logging, so I can query them there or export them to BigQuery for analysis. Many teams set up dashboards to continuously monitor BigQuery usage from audit logs.

44. Where can you find information about long-running queries in BigQuery?

There are a couple of places where I can check for long-running queries in BigQuery.

The first is the BigQuery console itself. Under the "Query History" or "Query Execution Details," I can see how long queries are running, how much data they scanned, and what stages are taking time.

The second is INFORMATION_SCHEMA.JOBS or INFORMATION_SCHEMA.JOBS_BY_PROJECT views. These system tables let me query metadata about jobs, including query text, start time, end time, user who ran it, and total bytes processed. If I want to specifically find queries that took longer than, say, 5 minutes, I can filter on the duration column.

Another place is Cloud Logging, where BigQuery automatically sends job information. If monitoring is enabled, I can create alerts to notify me when queries cross a certain runtime threshold.

In production, teams often build dashboards or alerts based on INFORMATION_SCHEMA or logs so that they don't have to manually search for long-running queries.

©shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

45. How can you track the state of a BigQuery job?

Every query or data load in BigQuery runs as a job, and each job goes through states like pending, running, or done. To track the state, I have a few options:

- BigQuery console: When I run a query in the UI, I can see its state in the Job History. It shows whether the job is still running or has finished, along with details like duration and bytes processed.
- INFORMATION_SCHEMA.JOBS views: I can run SQL queries against these views to check the state of jobs across a project. For example, I can filter on state to find jobs that are still running or check for errors in completed jobs.
- BigQuery API or client libraries: If I'm running jobs programmatically, I can use the API's jobs.get method to check the job's status. This is common in pipelines where I want to make sure a job finished successfully before moving to the next step.
- Cloud Logging: All job activity can also be tracked through logs, which can be useful for alerts.

So, in practice, if I want to monitor interactively, I use the console. If I want automated monitoring or to analyze trends, I use INFORMATION_SCHEMA or the API. This makes it easy to check whether jobs succeeded, failed, or are still in progress.

46. What's the key difference between a normal view and a materialized view in BigQuery?

A normal view in BigQuery is basically a saved SQL query. Every time I query the view, BigQuery executes the underlying SQL in real-time on the source tables. It doesn't store results; it's just a logical layer. That means normal views are always up to date, but they can be slower if the underlying query is heavy, because the computation happens fresh every time.

A materialized view, on the other hand, stores the results of the query physically. BigQuery automatically maintains this pre-computed result. So, when I query a materialized view, BigQuery can serve data much faster because it doesn't have to recompute everything from scratch.

The trade-off is: materialized views may not always reflect data in real-time (they are periodically refreshed), but they're excellent for performance and cost savings when I have repetitive analytical queries, like aggregations over large fact tables.

For example, if I have billions of transactions and I frequently query daily sales totals, a materialized view on SUM(amount) GROUP BY date will make queries almost instant compared to a normal view, which would re-scan the full transaction table each time.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

47. How does BigQuery handle automatic refresh of materialized views?

BigQuery takes care of refreshing materialized views automatically. It keeps track of changes in the base table and only updates the materialized view with the new or changed data. This happens incrementally and behind the scenes, so I don't have to manually trigger it.

The refresh process is very efficient instead of recomputing the whole query, BigQuery identifies which partitions of the base table have changed and only recomputes those. For example, if I have a materialized view on a partitioned transaction table, and new rows only came in for today's partition, BigQuery will only update that partition's result in the materialized view.

In practice, this makes materialized views cost-effective and near real-time for analytics. But I should know that refresh is not continuous; it happens periodically based on system-managed schedules. BigQuery guarantees that the data is updated within a few minutes, but it's not instant.

48. What is incremental refresh in materialized views?

Incremental refresh means that when the base table changes, BigQuery doesn't reprocess the entire table to update the materialized view. Instead, it only processes the new or modified portions of the base table and merges them into the existing results of the materialized view.

For example, imagine I have a materialized view calculating daily sales totals from a massive transactions table with billions of rows. If only today's sales data changes, BigQuery won't recalculate all previous days' totals again. It will only calculate for today and update the materialized view accordingly.

This incremental refresh is what makes materialized views highly efficient. It saves both time and cost. Without incremental refresh, every refresh would be as expensive as re-running the full aggregation query on the base table. With incremental refresh, I get near real-time updates at a fraction of the cost.

So in short, incremental refresh is the mechanism that ensures materialized views stay updated efficiently without re-scanning all data.

49. When will BigQuery use cached results?

BigQuery uses cached results when the exact same query has been run recently and the underlying data hasn't changed. The idea is to save time and cost by reusing the results that are already stored.

For example, if I run a query like SELECT COUNT(*) FROM sales WHERE date >= '2025-01-01', and then immediately run the same query again, BigQuery won't re-scan the table. Instead, it will pull the results from cache almost instantly, with no extra cost.

The main conditions for cache usage are:

- The query must be identical (same text, same settings).
- The source tables and referenced data must not have changed since the query was last run.
- The cached results are still within the 24-hour cache window.
- The query must not have certain flags disabled, like OPTIONS(disable_cache=true).

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

So, caching is most useful in development and in repeated reporting queries. But in production, I have to be cautious: if fresh data is coming in continuously, I might not want to rely on cache because it won't reflect the latest updates.

50. How does BI Engine accelerate queries for dashboards?

BI Engine is an in-memory analytics engine built into BigQuery. Its purpose is to make dashboard queries especially from tools like Looker Studio or Tableau much faster and more interactive.

What it does is cache frequently accessed tables or query results directly in memory (RAM). This means when a dashboard issues a query, instead of BigQuery going to the full columnar storage and scanning large amounts of data, BI Engine can serve the results from memory in milliseconds.

For example, if I have a sales dashboard that slices data by region and date, the underlying query might be scanning millions of rows. With BI Engine, those aggregations are preloaded and optimized in memory, so when the user switches a filter or changes the date range, the response feels instant, like using a traditional BI cube.

The main advantage is user experience: dashboards stay responsive even if the underlying tables are very large. The trade-off is BI Engine has size limits (the memory reservation), so it works best for high-value, frequently queried datasets rather than all data.

51. Name one limitation of materialized views in BigQuery.

One key limitation is that materialized views in BigQuery don't support all types of queries. They work mainly for relatively simple queries such as filters, projections, and aggregations on a single table. Complex queries involving joins, non-deterministic functions, or subqueries are not supported.

For example, if I want to create a materialized view that joins a transactions table with a products table, BigQuery won't allow it. I'd have to either simplify the query or create the materialized view only on one table.

This limitation exists because BigQuery optimizes materialized views with incremental refresh, and joins or complex logic make that much harder to maintain.

So, while materialized views are excellent for performance on large fact tables with aggregations, they can't always replace normal views or tables for more advanced queries

52. What file formats from GCS can be directly queried in BigQuery?

BigQuery supports external tables on top of files stored in Google Cloud Storage, and it can directly query several popular formats. The main ones are:

- CSV (comma-separated values)
- JSON (newline-delimited JSON)
- Avro
- Parquet
- ORC (Optimized Row Columnar)

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

Each format has its use case. CSV is simple and widely used but doesn't support schema evolution well. JSON is flexible for semi-structured data. Avro, Parquet, and ORC are columnar or binary formats that are more efficient for big data because they support compression and schema evolution.

For example, if I receive raw application logs in JSON format stored in GCS, I can create an external table pointing to those files and start querying them in BigQuery immediately, without having to load them. Similarly, if I have historical data stored as Parquet files, I can query them directly without extra storage costs in BigQuery.

So, BigQuery supports both text-based and columnar formats, giving flexibility depending on whether I prioritize simplicity (CSV/JSON) or efficiency (Avro/Parquet/ORC).

53. When would you use an external table on Bigtable in BigQuery?

I would use an external table on Bigtable in BigQuery when I want to analyze operational or time-series data stored in Bigtable using SQL, without copying or duplicating it.

Bigtable is great for high-throughput reads and writes, especially for use cases like IoT sensor data, clickstream logs, or financial tick data. But it's not designed for complex analytics. BigQuery is strong in analytics but isn't an operational database. External tables give me the best of both worlds: I can keep my operational workload in Bigtable, and still run ad-hoc analytics in BigQuery.

For example, if I have a Bigtable storing real-time IoT temperature readings from devices, and I want to run a one-off analysis like "average temperature per device per hour," I can define an external table in BigQuery that points to Bigtable and run SQL directly.

I'd use this approach when analytics is occasional and I don't want the overhead of exporting or syncing all Bigtable data into BigQuery. However, if I need frequent analytics on large volumes, I'd prefer to load or stream the data into BigQuery for better performance.

54. How does BigQuery connect with Cloud SQL for federated queries?

BigQuery supports federated queries, which means I can query data stored in Cloud SQL (MySQL or PostgreSQL) directly from BigQuery without moving it first.

The connection is set up using a **BigQuery connection resource**. In practice, I go to the BigQuery console, create a connection to my Cloud SQL instance (providing project, region, and database details), and grant BigQuery access to that database. Once the connection is in place, I can create external tables in BigQuery that point to Cloud SQL tables.

When I run a query, BigQuery pushes down as much of the filtering and aggregation as possible into Cloud SQL, then retrieves the results and integrates them with the rest of the query. This makes it possible to join Cloud SQL data with BigQuery tables in a single SQL query.

For example, if my Cloud SQL database has customer profile information, and my BigQuery dataset has transaction logs, I can query both together to answer questions like "total spend by customers in a certain region."

The important point is that Cloud SQL federated queries are best for small to medium datasets, because Cloud SQL isn't built for the same scale as BigQuery. For larger analytical workloads, I'd move the data into BigQuery.

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

55. What's a common use case for querying Google Sheets in BigQuery?

A common use case is when business teams or analysts keep small datasets in Google Sheets, like reference data, mapping tables, or manual inputs, and I want to join that with large datasets in BigQuery.

For example, suppose the finance team maintains a Google Sheet that maps campaign IDs to budget categories, and my BigQuery dataset has billions of rows of ad click data. Instead of uploading the sheet into BigQuery every time it changes, I can create an external table on top of the Google Sheet. This way, any updates the finance team makes are immediately reflected in BigQuery queries.

Other common use cases are:

- Storing look-up values or thresholds (like region-to-manager mapping).
- Keeping manually curated exception lists (like flagged customers).
- Prototyping and quick experiments where loading into BigQuery would be overkill.

So, Google Sheets federation is best for small, frequently updated datasets maintained by non-technical teams, which I can easily blend with large analytical datasets.

56. Why are federated queries usually slower than native BigQuery tables?

Federated queries are slower because the data doesn't live inside BigQuery's storage system. Instead, BigQuery has to reach out to an external source like Cloud SQL, Cloud Storage, Bigtable, or Google Sheets, read the data there, and then integrate it into the query.

The overhead comes from:

- Network latency: BigQuery pulls data from another system over the network, which takes time.
- Limited push-down: BigQuery tries to push filters and aggregations down to the external source, but not everything can be optimized. Sometimes, large amounts of raw data must be pulled first.
- External system limits: Systems like Cloud SQL or Sheets aren't designed for petabyte-scale analytics. They can become bottlenecks if the query asks for too much.
- No BigQuery-level optimizations: Features like partition pruning, clustering, or storage caching don't apply to external data.

So, while federated queries are great for convenience and small datasets, they're usually slower compared to native tables because the data isn't fully inside BigQuery's high-performance infrastructure.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

57. Why is data locality important when using federated queries?

Data locality is about making sure the external data source and BigQuery are in the same region. It's important because if they're in different regions, BigQuery has to move data across regions during the query, which adds latency, can make queries slower, and may even increase cost due to cross-region data transfer charges.

For example, if my BigQuery dataset is in us-central1 but my Cloud SQL instance is in europe-west1, every federated query will involve moving data between the US and Europe. This not only slows down the query but also risks higher costs and potential compliance issues (since some companies require data to stay in certain regions).

So, whenever I set up federated queries, I always make sure the external source (Cloud SQL, Bigtable, GCS, or Sheets) is in the same location as my BigQuery dataset. That way, the query runs faster and avoids unnecessary data transfer.

58. What is the default time travel retention period in BigQuery?

By default, BigQuery keeps a 7-day history of table data changes. This is called the time travel retention period. It means I can query the state of a table as it existed at any point within the past 7 days, even if rows have been updated or deleted.

For example, if I accidentally ran a query yesterday that deleted half my rows, I can use time travel to query the table as it looked before the delete and recover that data.

The default is 7 days, but in enterprise editions, the retention can be extended up to 90 days. This feature is extremely useful for recovering from mistakes or auditing changes without setting up manual backups.

59. How are table snapshots different from time travel queries in BigQuery?

Both snapshots and time travel give me access to historical versions of a table, but they work differently.

Time travel is temporary and query-based. It lets me query a past state of a table within the retention window (7 to 90 days). I don't create a new object; I just run a query with FOR SYSTEM_TIME AS OF and get the old data. But once the retention period passes, that version is gone.

A table snapshot, on the other hand, is a permanent, independent object that represents the state of a table at a specific point in time. Once I create a snapshot, it stays until I delete it, even beyond the normal retention period. Snapshots are useful if I want to "freeze" a table at a specific point for example, at the end of each month for reporting.

So the key difference: time travel is temporary and built-in, while snapshots are explicitly created and persist until I delete them.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

60. How can you recover a deleted table in BigQuery?

If I accidentally delete a table, I can still recover it using BigQuery's time travel feature, as long as I'm within the retention window (7 days by default). A deleted table is actually moved into a "soft delete" state, meaning BigQuery retains its data for that retention period.

To recover it, I can use the UNDROP TABLE statement or restore it by creating a new table from the old state using CREATE OR REPLACE TABLE ... AS SELECT * FROM table FOR SYSTEM_TIME AS OF ....

For example, if I deleted the sales table today, I can run a query pointing to the table as it existed yesterday and save it into a new table.

If I need a permanent safeguard beyond the retention period, I'd use scheduled snapshots or exports, but for accidental deletes, time travel recovery is usually enough.

61. What is a table expiration policy used for in BigQuery?

A table expiration policy in BigQuery is used to automatically delete tables or partitions after a certain period of time. This helps with cost control and lifecycle management of data.

For example, if I have a staging dataset where raw data is only useful for 30 days, I can set a default expiration policy of 30 days on that dataset. Any table I create inside it will automatically be deleted after 30 days, without me having to manually clean it up. Similarly, for partitioned tables, I can set partition expiration so that each partition is removed after a fixed duration (say, 90 days), keeping only recent data.

This is very useful in scenarios like logs, clickstream, or IoT data where data volume is massive, and older data doesn't need to be kept forever. Without an expiration policy, costs can grow quickly as data keeps piling up.

So, the purpose of table expiration is to automate cleanup, save storage costs, and ensure only relevant data is kept in BigQuery.

62. When does BigQuery start charging long-term storage rates?

BigQuery automatically reduces storage costs once a table has not been modified (no writes, updates, or appends) for 90 consecutive days. After this 90-day period, the data is considered "long-term storage," and the cost per GB is about half the normal storage rate.

The important point is that long-term storage is not a different storage class like in GCS it's the same BigQuery storage, but discounted once data becomes inactive. If I query the table, I still pay normal query costs, but the storage cost is lower.

If I write to the table again (insert, update, append), the 90-day timer resets, and the table goes back to active storage pricing until it becomes idle again.

So in short: long-term storage kicks in automatically after 90 days of no changes, giving me cost savings for historical data that I only query occasionally.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

63. What is a common pitfall of using long-term retention with very large datasets?

A common pitfall is assuming that long-term storage reduces all costs, but in reality, it only reduces the storage cost not the query cost. With very large datasets, the biggest cost often comes from scanning data during queries. If I run queries that touch entire historical tables, I'll still pay high query costs even if the storage is cheaper.

For example, if I have 200 TB of log data in long-term storage, my storage bill might be low, but if analysts run queries without proper partition filters (like forgetting to filter by date), BigQuery will scan the whole 200 TB. That can make query costs skyrocket.

Another pitfall is forgetting that writing to a table resets the 90-day clock. So if I occasionally append new rows to a massive table, it never qualifies for long-term pricing, even though most of the data is old. A better design would be to use partitioned tables so only the "active" partitions get updates, and older partitions continue to enjoy long-term pricing.

So the big trap is thinking "long-term storage solves my cost problem" without realizing query design and partitioning are just as important.

64. What are query stages in BigQuery?

When BigQuery executes a query, it doesn't run it as one big block. Instead, it breaks the query down into multiple stages. Each stage represents a logical step in the execution plan things like scanning the source tables, applying filters, shuffling data across workers, performing joins, aggregations, and finally writing the output.

For example, if I run a query that filters sales data, joins it with customer data, and then calculates total sales per region, BigQuery might create:

- One stage to scan the sales table,
- One stage to scan the customer table,
- A shuffle stage to prepare matching keys,
- A join stage,
- An aggregation stage,
- And a final stage to write the results.

Each stage can run in parallel across many workers. That's how BigQuery achieves massive scalability.

As an engineer, looking at query stages in the execution details is very useful for troubleshooting. If one stage takes much longer than the rest, that's usually where the bottleneck is for example, a large join or shuffle. So, query stages give me visibility into how BigQuery is internally processing my SQL.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

65. What is the role of slots in BigQuery query execution?

Slots are the units of computational capacity in BigQuery. Think of a slot as a virtual CPU plus memory that BigQuery uses to execute query tasks.

When a query runs, BigQuery assigns slots to it. The more slots it has, the more tasks can run in parallel, and the faster the query finishes. For example, if a query has a scan stage that can be split into 1,000 parallel tasks, and I have enough slots, those tasks can all run at once. If I have fewer slots, the tasks will be queued and executed in batches, making the query slower.

In on-demand pricing, I don't directly manage slots BigQuery dynamically allocates them. But in flat-rate or enterprise editions, I can reserve slots and allocate them to different projects or workloads. This gives me predictable performance and cost control.

So, slots are the backbone of query execution. They decide how much parallelism I get, and understanding them helps me design queries and capacity planning for large workloads.

66. Why are shuffle operations expensive in BigQuery queries?

Shuffle operations happen when BigQuery has to redistribute data across workers so that related rows end up together. This usually happens during joins, aggregations, or DISTINCT queries.

For example, if I join a sales table on customer_id, BigQuery needs to make sure all rows with the same customer_id are on the same worker before it can perform the join. To do that, it shuffles data across the network.

Shuffles are expensive because:

- They involve network transfer of large amounts of data between workers.
- They require disk I/O if the shuffled data is too large to fit in memory.
- They can cause skew issues if one key is much larger than others, one worker gets overloaded while others stay idle.

As a result, shuffles often show up in the execution plan as the slowest stage.

To reduce shuffle costs, I try to design queries carefully for example, by filtering data early, partitioning/joining on evenly distributed keys, and avoiding unnecessary DISTINCT or big CROSS JOINs.

In short, shuffles are expensive because they break the flow of parallelism and force heavy network and disk operations.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

67. What does it mean when a BigQuery query "spills to disk"?

When a query "spills to disk" in BigQuery, it means the amount of data being processed in a stage is too large to fit into the memory assigned to the slots. Instead of failing, BigQuery writes part of that intermediate data to temporary storage (disk) and then reads it back later to continue processing.

For example, if I run a query with a very large join or aggregation that produces intermediate results bigger than slot memory, BigQuery can't keep all of it in RAM. So it spills to disk.

While spilling prevents queries from failing due to memory limits, it makes queries slower. Disk I/O is much slower than in-memory operations, and if the spill is large, query performance can degrade significantly.

In practice, if I see queries spilling to disk in execution details, that's a signal I should optimize maybe by filtering earlier, reducing the amount of data shuffled, or using partitioned joins.

68. What's the difference between broadcast join and shuffle join in BigQuery?

The difference comes down to how BigQuery distributes the data when performing the join.

- Broadcast join: BigQuery takes the smaller table and broadcasts (copies) it to all workers. Then each worker can join its portion of the big table with the full small table locally. This avoids heavy shuffling because only the small table is replicated, not the large one. Broadcast joins are very fast when one table is tiny compared to the other.
- Shuffle join: When both tables are large, BigQuery can't broadcast either one. Instead, it redistributes (shuffles) rows from both tables across workers based on the join key, so that matching keys land on the same worker. Then it performs the join. Shuffle joins are much more expensive because they involve network transfer and disk usage.

For example, if I join a 10 GB sales table with a 1 MB customer lookup table, BigQuery will use a broadcast join. But if I join two 10 TB tables, it will use a shuffle join.

So, broadcast joins are optimized for big-small joins, shuffle joins are used when both sides are large.

69. Why do queries behave differently for small vs very large datasets in BigQuery?

Queries behave differently mainly because BigQuery's execution engine is optimized for distributed parallelism, and the scale of data changes how execution plans are chosen.

On small datasets, BigQuery can often process the query entirely in memory without spilling or shuffling much. Operations finish quickly, and optimizations like broadcast joins are used. For example, a query on a few thousand rows might return almost instantly.

On very large datasets, the engine has to scan terabytes or petabytes of data. That means more shuffle operations, higher risk of spilling to disk, and stricter reliance on partitions and clustering for efficiency. Also, BigQuery might switch from broadcast joins to shuffle joins, or break queries into more stages.

Another difference is **cost impact**. For small datasets, the query cost is minimal, but for very large datasets, forgetting to apply filters or partitions can cause massive scans and much higher cost.

So, the behavior difference is really about scale: BigQuery adjusts its strategies based on data size. That's why queries that run fine on small test datasets sometimes behave very differently and slower when run on production-scale data.

© Shubham Wagekar

70. Why is Dataflow often used for ETL before loading data into BigQuery?

Dataflow is often used for ETL into BigQuery because it can handle large-scale, complex data transformations in a streaming or batch mode before the data lands in BigQuery.

Some reasons why Dataflow is a good fit:

- Scalability: Dataflow is based on Apache Beam, so it can scale automatically to handle terabytes of data. Whether I'm processing historical batch data or streaming real-time events, it can manage the load.
- Transformations: It allows me to clean, enrich, and transform data in-flight. For example, parsing nested JSON, flattening records, joining multiple sources, applying business logic, and then outputting clean, analytics-ready data into BigQuery.
- Streaming pipelines: With streaming mode, Dataflow can ingest real-time events (e.g., from Pub/Sub) and load them continuously into BigQuery, giving near real-time analytics.
- Integration: Dataflow has built-in connectors to read from many sources like Pub/Sub, Cloud Storage, or Bigtable and then write directly into BigQuery.

For example, in an e-commerce system, raw clickstream logs might be collected in Pub/Sub. Dataflow can process them in real time (filtering bots, enriching with geo-data), then load the clean data into BigQuery for dashboards.

So, Dataflow acts as the transformation layer, ensuring BigQuery only stores clean and structured data ready for analytics.

71. How can Dataproc be integrated with BigQuery for Spark/Hive workloads?

Dataproc is Google's managed Hadoop and Spark service, and it integrates with BigQuery so I can use my existing big data tools while still taking advantage of BigQuery's storage and analytics.

There are two main ways of integration:

1. BigQuery connector for Spark/Hive: With the connector installed on Dataproc, Spark or Hive jobs can read from BigQuery tables as if they were normal datasets and write results back to BigQuery. This is useful when I already have Spark code written and want to reuse it without moving to Dataflow or rewriting in SQL.
2. Data export and import: Spark jobs can process raw data (say, from HDFS or GCS) and output results into BigQuery tables for further analysis.

A common use case is when a company has existing Spark-based ETL jobs (like complex machine learning feature engineering or advanced transformations). Instead of rewriting everything in SQL or Beam, I can run those Spark jobs in Dataproc and push the results into BigQuery for downstream analytics and reporting.

So, Dataproc lets me bridge the gap between traditional big data processing (Spark/Hive) and modern analytics in BigQuery.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

72. How does Cloud Composer help orchestrate BigQuery jobs?

Cloud Composer is a managed service for Apache Airflow, and its main role is orchestration making sure workflows run in the right order and at the right time. For BigQuery jobs, Composer helps by managing dependencies, scheduling, and monitoring.

For example, if I have a pipeline where raw data first arrives in Cloud Storage, then needs to be loaded into a staging table, cleaned with a transformation query, and finally aggregated into a reporting table, Composer can orchestrate the whole flow. It can wait for the load job to finish before triggering the transformation, and then run the final aggregation after that.

Composer provides operators like **BigQueryOperator** or **BigQueryCheckOperator**, which let me run queries, load jobs, or even validate data directly inside workflows. I can also integrate other services, like triggering a Dataflow job first and then a BigQuery job.

In real-world setups, I've seen Composer used to coordinate daily ETL pipelines, ensure retries on failed BigQuery jobs, and notify teams if something breaks. It essentially automates and governs the flow of BigQuery workloads in larger data pipelines.

73. How do Pub/Sub and BigQuery work together in real-time pipelines?

Pub/Sub is Google's messaging service for streaming data, and it works really well with BigQuery to build real-time analytics pipelines. The typical setup is:

- Events (like clicks, IoT readings, or transactions) are published to a Pub/Sub topic.
- A consumer, usually Dataflow, reads from Pub/Sub, processes or enriches the events, and writes them into BigQuery.
- BigQuery then stores the data so analysts and dashboards can query it almost in real time.

For example, in an e-commerce app, every user click or purchase can be sent to Pub/Sub immediately. Dataflow consumes these events, cleans them (e.g., removes invalid records, adds geo lookup), and streams them into a BigQuery table. A dashboard connected to BigQuery can then show "orders in the last 5 minutes" live.

So, Pub/Sub acts as the ingestion layer, Dataflow does the processing, and BigQuery is the analytics layer. Together, they form the backbone of real-time pipelines.

74. How does BigQuery integrate with Looker for BI reporting?

Looker connects to BigQuery natively through the BigQuery connector. Since BigQuery is serverless and can handle SQL at scale, Looker doesn't need to extract data it pushes SQL queries down to BigQuery and retrieves only the results.

In practice, Looker models are written in **LookML**, which defines dimensions, measures, and joins. These get translated into SQL that runs directly on BigQuery. This means all the heavy lifting (aggregations, joins, filters) happens inside BigQuery, not in Looker.

For example, if I build a sales dashboard in Looker, every filter or chart is really generating a SQL query that runs in BigQuery. The performance comes from BigQuery's scale, and the visualization layer is Looker.

The benefit is I don't duplicate data I just keep it in BigQuery and expose it via Looker. Also, I can leverage BigQuery features like BI Engine for in-memory acceleration, which makes Looker dashboards even more responsive.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

75. How can BigQuery ML be used as part of a pipeline?

BigQuery ML allows me to build and use machine learning models directly in SQL, without moving data out of BigQuery. It can fit naturally into data pipelines.

For example:

1. Training step: I can train a model using SQL like CREATE MODEL on my historical data in BigQuery. This could be anything from linear regression to classification or even boosted trees.
2. Prediction step: Once the model is trained, I can use it in the pipeline to score new incoming data with ML.PREDICT. Predictions can be written into a new table or joined with existing data.
3. Orchestration: With Composer or Dataflow, I can schedule model retraining on fresh data, then run predictions regularly or in near real time.

A practical use case is churn prediction. I could train a churn model on past customer behavior in BigQuery. Then, as new customer activity data comes in (daily or hourly), my pipeline can use the trained model to predict churn risk and output scores into a BigQuery table. That table can then feed dashboards or even trigger alerts.

So, BigQuery ML makes ML part of the data pipeline without the overhead of exporting data to external ML platforms, which keeps the pipeline simpler and faster.

76. How can BigQuery be used as both a data lake and a warehouse?

BigQuery is flexible enough to act as both a data lake and a data warehouse, depending on how it's used.

- As a data lake: BigQuery can query data directly from external storage like Google Cloud Storage (CSV, JSON, Avro, Parquet, ORC). I don't need to load it first; I can use external tables or BigLake tables. This means raw, unstructured, or semi-structured data can sit in GCS, but I can still query it with SQL. BigQuery also supports nested and repeated fields, which makes it good for semi-structured JSON-style data.
- As a warehouse: Once the data is cleaned, structured, and loaded, BigQuery provides all the features of a warehouse partitioning, clustering, materialized views, BI integrations, and performance optimizations. Analysts can run complex queries at scale with very fast results.

For example, logs might be stored in GCS (lake), and BigQuery can query them directly for ad-hoc exploration. Then, the cleaned and aggregated results are stored in BigQuery tables for dashboards and reporting (warehouse).

So, BigQuery can serve both roles: raw data exploration (lake) and structured analytics (warehouse), which is why many companies simplify their architecture by using BigQuery as a "lakehouse."

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

77. What is the role of BigLake in modern data architectures with BigQuery?

BigLake is Google's unified storage engine that extends BigQuery's capabilities beyond just BigQuery-managed tables. Its role is to let me access and secure data across multiple storage systems not just BigQuery storage, but also files in GCS and even data in other lakes with a consistent security and governance layer.

The main role BigLake plays is unification:

- I can create BigLake tables pointing to Parquet or ORC files in GCS and query them in BigQuery as if they were native tables.
- I can apply the same fine-grained access controls (row-level, column-level security) on those external tables.
- It allows organizations to avoid duplicating data they can leave raw data in open formats in GCS but still query it through BigQuery.

For example, if my data science team wants to use raw Parquet data in GCS with Spark, and my BI team wants to query it in BigQuery, BigLake lets both teams work on the same data without copies.

So, BigLake acts as the bridge that brings data lake flexibility into the BigQuery analytics ecosystem, making it easier to build lakehouse architectures.

78. How do you handle late-arriving data in BigQuery streaming pipelines?

Late-arriving data is common in streaming for example, IoT devices that go offline and send events later, or logs that arrive out of order. In BigQuery, handling this usually involves partitioned tables and streaming inserts with late data support.

Strategies I'd use:

- Partition by event timestamp: I store data by the event time, not the ingestion time. That way, if an event arrives 2 hours late, it still goes into the correct partition.
- Allow updates with MERGE: I can use scheduled queries or Dataflow jobs to reconcile late data, merging it into the right partitions.
- Partition expiration buffers: I avoid expiring partitions too quickly. For example, if I expect data to be late by up to 7 days, I keep the partitions alive longer.
- Deduplication: Sometimes late data causes duplicates. I handle this by using unique IDs and de-duplication logic in streaming pipelines before writing to BigQuery.

So, the key idea is to design pipelines with the expectation of late data and ensure partitioning and reconciliation processes account for it.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

79. How does BigQuery enforce row-level and column-level security for compliance?

BigQuery provides fine-grained security controls to meet compliance needs:

- **Row-level security**: I can define policies that filter rows based on conditions. For example, if analysts in Europe should only see rows where region = 'EU', I create a row-level security policy tied to their IAM identity. When they query the table, BigQuery automatically applies the filter, so they only see the rows they're allowed to.
- **Column-level security**: I can restrict access to specific sensitive columns, like salary or ssn. This is done using policy tags in Data Catalog. For example, I can tag the ssn column as "Restricted," and only users with the right permissions can access it. Other users querying the table won't even see that column.

Both controls integrate with IAM, so access is consistent across the organization.

For example, in a healthcare dataset, I might restrict doctors to only see patient data from their hospital (row-level), and only the compliance team can see sensitive identifiers like social security numbers (column-level).

So, these features allow BigQuery to enforce compliance rules without creating multiple copies of the same data, which keeps things simpler and safer.

80. When would you choose a multi-region dataset in BigQuery?

I would choose a multi-region dataset when my data needs to serve users or applications across multiple regions, or when I want higher availability without managing regional replication myself.

Multi-region datasets store data across multiple regions within a broader geographic area (like "US" or "EU"). If one region has an outage, queries can still run from another region in that multi-region area.

A good use case is when my company has users across the United States, and both East and West Coast teams query the same dataset. By keeping the dataset in the "US" multi-region, I don't have to worry about latency differences or replication. Another case is if I rely heavily on other Google products (like Data Studio or Looker) that expect "US" or "EU" datasets for compliance reasons.

So, I'd choose multi-region when I want resilience and geographic redundancy, or when my organization has compliance requirements to keep data in a broader location like US or EU.

81. How can datasets be securely shared across projects in BigQuery?

Datasets in BigQuery can be shared securely by using IAM roles at the dataset or table level. Instead of moving or copying data, I can grant access to specific users, groups, or service accounts in another project.

For example, if I want a data science team in a different project to access a dataset, I can assign them roles like roles/bigquery.dataViewer (read access) on the dataset. This way, they can query the data directly without creating duplicates.

For finer control, I can also share only specific views, not the entire dataset. This way, sensitive columns or rows can be hidden, and only the necessary fields are exposed.

Another secure approach is using Authorized Views or Row/Column-level security with policy tags. This ensures compliance when sharing sensitive datasets.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

So, the key idea is that I don't move data between projects I use IAM permissions and security features to provide controlled access to the right users.

82. How can BigQuery support a data mesh architecture?

In a data mesh, different domains (like sales, marketing, finance) own their data and publish it as "data products" that others can consume. BigQuery supports this because it allows decentralized ownership but centralized discoverability.

Each domain team can have its own dataset in its own project, with full control over schema, policies, and refresh schedules. They can then share that dataset with other teams via IAM, authorized views, or Data Catalog tags.

For example, the marketing team may own a marketing_campaigns dataset, while finance owns a finance_transactions dataset. Both publish their data products in BigQuery, and analysts or other teams can query them together when needed.

BigQuery also integrates with Data Catalog, so these datasets can be registered, tagged, and discovered across the organization, which is a key part of making a data mesh work.

So, BigQuery enables data mesh by letting domain teams own their datasets, enforce their own governance, but still make them discoverable and consumable at scale.

83. Name one best practice for cost optimization in BigQuery queries.

A very important best practice is always avoid SELECT * on large tables. Instead, query only the columns you need.

This is because BigQuery charges based on the amount of data scanned, not the number of rows returned. If I run SELECT * on a 10 TB table when I only need 3 columns, I'll be charged for scanning the entire 10 TB instead of just a few gigabytes.

In production, I often enforce this by creating views or using column-level security so analysts can't accidentally query unnecessary columns. Another tip is to use partition filters for example, always filtering on event_date in a partitioned table.

So, minimizing scanned data by selecting only required columns is one of the simplest and most effective ways to save cost in BigQuery.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

84. How do you design pipelines in BigQuery to meet strict SLAs?

When I need strict SLAs (like dashboards must be updated within 5 minutes or daily jobs must finish before 6 AM), I design my pipelines carefully:

1. Partitioned and clustered tables: So queries scan only the necessary slices of data and run faster.
2. Materialized views or scheduled queries: Pre-compute heavy aggregations instead of recalculating every time.
3. Slot reservations: In enterprise setups, I reserve dedicated slots for critical workloads, so they don't compete with other jobs.
4. Orchestration with Composer: Use Cloud Composer (Airflow) to manage dependencies, retries, and alerting. If one step fails, it retries automatically, keeping the pipeline reliable.
5. Monitoring: Set up query performance monitoring (via INFORMATION_SCHEMA or Cloud Logging) to detect bottlenecks early.
6. Decoupling layers: I use staging, transformation, and reporting layers so issues in raw ingestion don't directly impact dashboards.

For example, if I need a marketing dashboard refreshed every hour, I might set up streaming ingestion for real-time data, pre-aggregate it into hourly materialized views, and guarantee slot reservations so the job always finishes on time.

So, meeting SLAs in BigQuery is about combining smart table design, orchestration, and capacity management.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar